React Session — Date :- 8/10/26

React Form :-

ES6 Features of JS :-



1) setTimeout
2) setInterval
3) Event loop
4) Call back queue
5) Asynchronous Proy
6) Promises
7) async await
8) Form

setTimeout :-

```
console.log("Hello")
const t1 = setTimeout("c.log('Hi1')", 1000)
const t2 = setTimeout("c.log('Hi2')", 2000)
c.log(t1)
c.log(t2)
```

	Expected O/P	Actual O/P
↓	Hello	Hello
	Hi1	1
	Hi2	2
	1	Hi1
	2	Hi2

```
Array.map((e) => { c.log(- - )
```

Higher Order Fun?

Callback Fun?

* setTimeout :- It is asynchronous JS browser fun?. And it is also a higher order fun?.
  - It wait to execute or delay execution of callback fun? up to whichever time we mentioned as timeout.
* callback queue :- It's manage the execution of callback fun? of setTimeout
* Event Loop :- Check the if any callback is in waiting callback queue then

they check if call stack is empty or not. if call stack is empty then they pass this fun^n to call stack for execution.

* call stacks :- Manage the execution of all fun^n execution context.

Form :- form in react

- special library → React hook form

Process :-

1) Form creation <Form

2) Register the text field to form

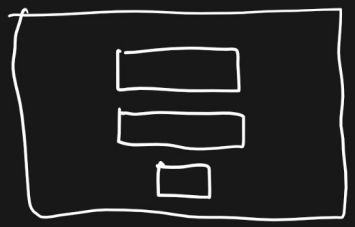
3) Handling form submit

4) Add validation

5) Error Handling

6) CSS part for form → H.W.

7) prevent multiple submit



```
let x=10
function blockingCodeForFewSec()
{
   for(let i=0;i<100000000;i++)
   {}
   console.log("From function")
}
setTimeout(()=>{
   console.log("Timer 1 done");
},5000)

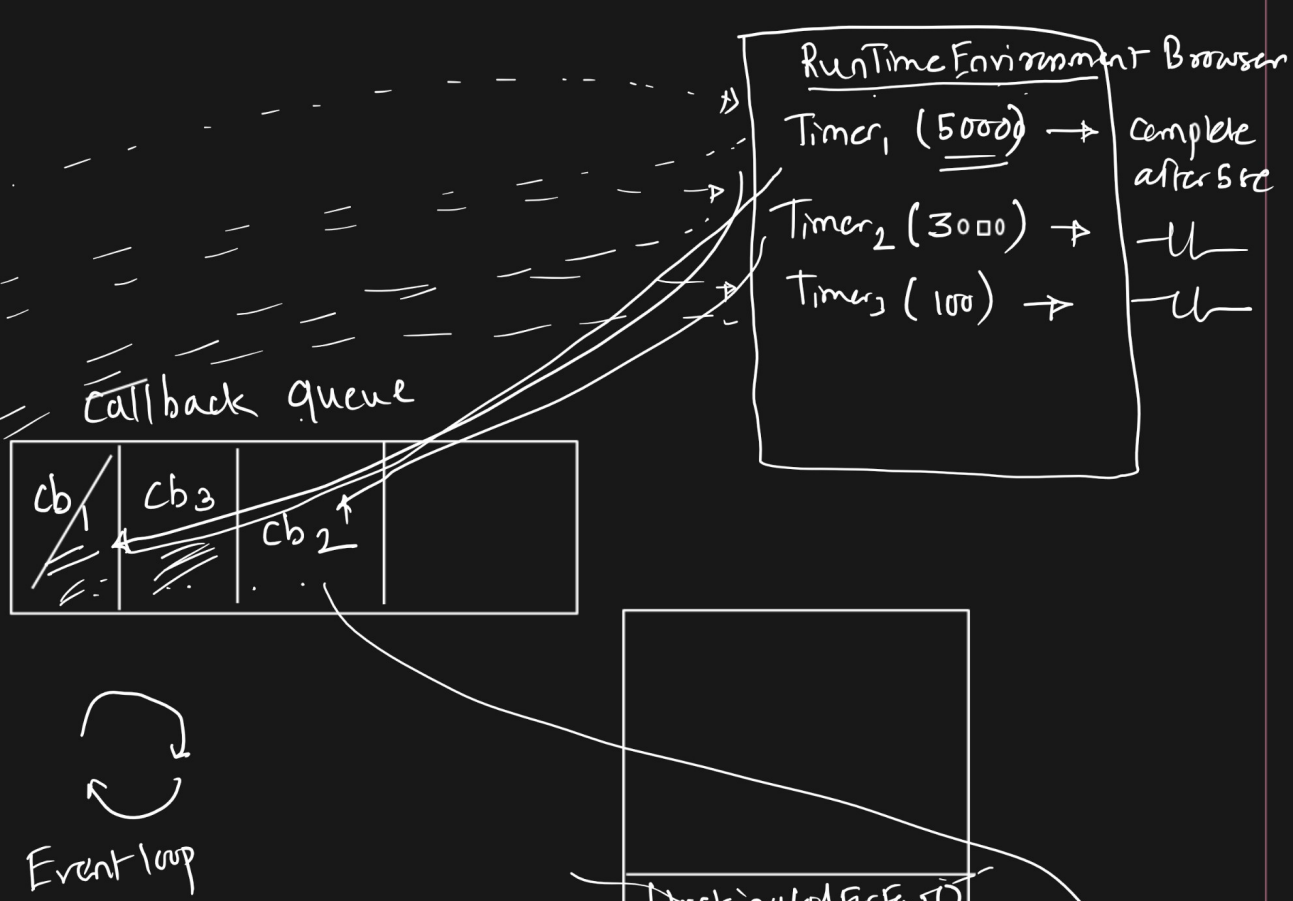
blockingCodeForFewSec()

setTimeout(()=>{
   console.log("Timer 2 done")
},3000)

setTimeout(()=>{
   blockingCodeForFewSec()
   x++;
   console.log(x)
},100)

console.log(x)
```

11 sec



Console.log

x = 10

Timer 1 done

Timer 3 done

Timer 2 done

1 2 3 4 5 6 7 8 9 10 11

Timer2 1 2 3

✓ Timer 3 100 msec

Call back fun → Disadvantage

1) call back hell
2) Inversion of Control

} To handle that issues "Promises" by

Call back hell = = → call back fun

Difficult to handle
as well as to
read also it is
difficult

```
() => {
    () =>
    {
    }
}
() => {
    () => { }
}
```

